SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3

Sl. No	Questions	Bloom's Level
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3

Sl. No	Questions	Bloom's Level
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

*I **B Ragava Prasath (192421026)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

B Ragava Prasath

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWERS

Question 1

Testing as an engineering activity: Testing is not random clicking; it is a planned, measurable and repeatable engineering discipline with defined inputs (requirements, design), activities (planning, test design, execution, reporting) and outputs (test cases, defect reports, metrics). The team must move from 'try and see' to engineered testing.

Introducing systematic design, tools and standards: Design: derive test cases from requirements using a traceability matrix. **Tools:** test management (TestLink/Jira-Xray), defect tracking (Jira/Bugzilla), automation (Selenium, JUnit), API testing (Postman). **Standards:** IEEE 829 test documentation, ISO/IEC 29119, a common test-case template and entry/exit criteria.

Improvement in defect detection and quality: Every requirement is covered by at least one test, so gaps are visible; defects are found earlier and are reproducible; regression suites stop old defects returning; metrics (defect density, coverage) guide decisions.

Techniques replacing ad-hoc testing: Equivalence class partitioning, boundary value analysis, decision tables, state-transition testing, use-case based testing, plus risk-based prioritisation and a regression suite.

Practical example: Funds transfer: requirement 'minimum Rs.100, maximum Rs.50,000 per transaction'. Boundary tests: 99, 100, 101, 49,999, 50,000, 50,001; equivalence classes: valid, below minimum, above maximum, non-numeric; decision table for balance sufficient/insufficient and account active/blocked. Ad-hoc testing would probably have tried only one or two amounts and missed the 50,000 limit defect.

Question 2

Approach: Adopt a test process aligned with the V-model so that every development phase has a matching test activity.

Stages and relevance: Requirements: review requirements for testability, prepare acceptance test plan. **Design:** review design, prepare system and integration test cases. **Coding:** unit testing, static code review. **Integration:** test interfaces (card reader, keypad, bank server, cash dispenser). **System testing:** functional, performance, security and recovery tests. **Acceptance:** user/bank validation. **Maintenance:** regression testing after changes. Each stage catches defects where they are cheapest to fix.

Applying to ATM withdrawal: Requirement: withdrawal allowed only if PIN valid, balance sufficient, and amount is a multiple of 100 within the daily limit. Test cases: valid PIN and balance (cash dispensed, balance debited); wrong PIN three times (card blocked); amount 150 (rejected); amount above balance (rejected); network failure after debit (transaction must roll back/reverse); cash-dispenser empty.

Process consistency reduces defects: The same entry/exit criteria, templates and environments give the same results every time, so results are comparable, reproducible and auditable; misunderstandings and missed steps decrease.

Expected outcomes: Consistent and reliable results, earlier defect detection, lower rework cost, fewer production failures, traceability from requirement to test result, and higher customer trust.

Question 3

Definitions: Error (mistake): a human action producing an incorrect result. **Defect (fault/bug):** the flaw in the code or document caused by the error. **Failure:** deviation of the observed behaviour from the expected behaviour when the defect is executed.

Relationship: Error -> Defect -> Failure. A developer misunderstands the rule (error), writes 'if (attempts > 3)' instead of 'if (attempts >= 3)' (defect), and the account is not locked on the third wrong password (failure).

Scenario examples: Login: valid credentials rejected because the password comparison is case-sensitive by mistake. Transaction: amount 1,000.50 stored as 1,000 because of integer typecasting (defect), and the user sees a wrong balance (failure).

Effect of poor understanding: Testers may report symptoms without cause, developers may fix the wrong thing, defects may be misclassified, duplicate or rejected, and root-cause analysis and prevention are not possible.

Improving defect-report clarity: Use a standard report: ID, title, module, steps to reproduce, test data, expected vs actual result, environment/build, severity, priority, screenshots/logs, status. Example title: 'Login fails for valid user with upper-case e-mail', rather than 'Login not working'.

Question 4

Relevant principles: Testing shows presence of defects; exhaustive testing is impossible; early testing; **defect clustering** (about 80% of defects lie in 20% of modules - Pareto); pesticide paradox; testing is context dependent; absence-of-errors fallacy.

Guiding test planning: Use defect clustering to allocate more effort to the high-defect modules; use early testing to catch them in reviews; refresh test cases regularly to counter the pesticide paradox; accept that exhaustive testing is impossible and therefore select tests by risk.

Prioritising test cases: Rank by (a) business criticality, (b) defect history of the module, (c) complexity and change frequency, (d) usage frequency. Run high-priority cases first and in every regression cycle; low-risk cases are run less often.

Efficiency gains: More defects found per hour, fewer delays, optimal use of limited time and testers, better risk coverage.

Real-time example: In an online-shopping project the payment and cart modules produced 70% of the defects. The team added extra boundary, integration and regression tests for these modules, ran them daily, and did lighter testing of the static 'About us' pages. Release delays dropped and escaped defects reduced.

Question 5

How defects originate: Requirement ambiguity, design flaws, coding mistakes, incorrect configuration, poor test data, lack of reviews, and changes made without impact analysis.

Categorisation: By origin/phase (requirements, design, coding, testing), by type (logic, interface, data, algorithm, documentation), by severity (critical to minor), by priority and by status. Classes follow the defect taxonomy of Beizer/IEEE 1044.

Using repository data: Analyse trends by module and defect class; derive checklists from frequent defect types; design new test cases for recurring patterns; feed lessons into code reviews and developer training; perform root-cause (Pareto/fishbone) analysis.

Improvements to avoid recurrence: Mandatory root cause field for each defect, periodic repository review meetings, regression tests added for every fixed defect, searchable repository with consistent classification.

Expected quality improvements: Reduced repeat defects, lower defect density across releases, faster triage, better test coverage of weak areas and more predictable releases.

Question 6

Root-cause identification: Reproduce the crash with exact steps; study logs/crash reports (Logcat, Crashlytics); isolate by layer: UI input, API call, server response, database. Typical causes: null or unvalidated input, unhandled exception on timeout, memory leak, incompatible OS/device, concurrent transactions.

Principles applied: Early testing, defect clustering (fund-transfer module is high risk), exhaustive testing is impossible so use risk-based tests, and testing is context dependent (mobile network and device variety).

How systematic testing reduces crashes: Planned test cases cover valid and invalid inputs, boundaries, interruptions (call, low battery, network loss), device/OS combinations and load, so crashes are discovered before release.

Examples: Enter amount 0, negative, very large, or letters; switch off network after tapping 'Confirm'; transfer twice quickly (double submit); rotate the screen during OTP entry; test on low-memory devices.

Improvements: Input validation and exception handling, automation of regression tests, device-cloud testing, performance and stress tests, crash analytics monitoring and a root-cause review after every production incident.

Question 7

Testing: A planned activity to execute software with the intention of revealing defects and to evaluate quality. It is performed mainly by testers, can be planned and automated, and ends with a defect report.

Debugging: The activity of locating the cause of a known failure and fixing it. It is performed by developers, is investigative, and ends with a code correction.

Roles in quality: Testing detects and measures quality; debugging repairs and improves it. Re-testing and regression testing verify the fix.

Effect of improper usage: If developers 'debug' instead of testing, defects are found only by chance and with no records; if testers try to fix code, independence is lost. Failures go undocumented, fixes introduce new defects and the same defects reappear.

Integrated approach: Test -> report defect (steps, logs) -> developer debugs and fixes (locate, correct, unit-test) -> tester performs confirmation testing -> regression testing -> close defect. Keep clear ownership, a shared defect tracker and a definition of done.

Question 8

Strategy: Identify target browsers/versions/devices from analytics (Chrome, Firefox, Safari, Edge; desktop and mobile); build a compatibility matrix; prioritise critical flows (search, cart, checkout, payment); run functional and UI tests on every combination; automate repeated checks.

How defects originate: Differences in rendering engines (Blink, Gecko, WebKit), unsupported CSS/HTML5/JavaScript features, vendor-specific behaviour, screen resolution, cookie and security policies, and unpolyfilled code.

Systematic testing improves reliability: Consistent behaviour on all supported browsers, fewer customer complaints and abandoned carts, and early detection of environment-specific defects.

Tools and techniques: Selenium Grid/WebDriver, BrowserStack, Sauce Labs, LambdaTest, Cypress/Playwright, Chrome DevTools device mode. Techniques: compatibility matrix with pairwise (combinatorial) selection, visual regression and screenshot comparison, responsive layout testing, HTML/CSS validators.

Question 9

Categorising the defects: By type: **algorithmic/logic defects** (wrong formula for HRA or tax), **control-flow defects** (wrong condition for slab), **data defects** (rounding, wrong data type), **boundary defects** (salary exactly at slab limit). By origin: coding phase. By severity: high (financial and legal impact).

How understanding classes improves testing: Each class suggests a technique: logic defects - decision-table and path testing; boundary defects - boundary value analysis; data defects - data-type and rounding tests; so test design is targeted and defect-prone areas are covered.

Prevention in future releases: Code reviews and walkthroughs of formulas, unit tests for every calculation rule, shared constants for tax slabs, checklists built from past defect classes, and parallel runs against manual calculations.

Testing-strategy improvements: Create a test-data set with slab boundaries (e.g. 2,50,000 / 2,50,001), negative and zero values, leave and overtime combinations; add automated regression for payroll rules; add independent verification by finance users; track defect classes in the repository.

Question 10

Testing as engineering: Apply engineering principles: analyse requirements, design test cases, define expected results, execute, record, measure and improve.

Importance of structured test case design: Gives coverage of requirements, repeatability, traceability, reusability, ability to estimate effort and to review the tests, and objective pass/fail decisions.

Improvement in defect detection: Random tests repeat common paths and miss unusual combinations. Designed tests deliberately target boundaries, invalid inputs and combinations, so more defects are found with fewer tests. Example: for age field 18-60, structured tests 17, 18, 19, 59, 60, 61 find an off-by-one error that random inputs like 25 or 40 will never reveal.

Techniques: Equivalence partitioning, boundary value analysis, decision tables, state-transition testing, cause-effect graphing, error guessing (as a supplement), use-case testing, and peer review of test cases with a requirements traceability matrix.

Question 11

Analysis using principles: Early testing, defect clustering at module interfaces, and context dependency (patient data is safety-critical) indicate focusing on integration and data-flow testing.

How defects originate: Mismatched data definitions between modules, duplicate storage, missing validation, unsynchronised updates, wrong data mapping or format (date, units), concurrency issues, and incorrect database transactions.

Systematic testing for data integrity: Integration tests following the data end-to-end (registration -> doctor -> pharmacy -> billing); CRUD and database consistency checks; boundary and format validation; transaction rollback tests; concurrency tests; data migration reconciliation.

Example: Patient allergy updated in the clinical module but the pharmacy module shows old data. A test that updates in one module and verifies in all others exposes the stale-data defect.

Improving defect tracking and reporting: Log module, data record IDs, and before/after values; link defects to requirements; use severity ratings that prioritise patient safety; weekly defect review and trend reports.

Question 12

Analysing the issue: Repeated defects indicate that lessons learned are not feeding back into test design: missing regression tests for fixed defects, poor root-cause analysis, weak classification, and tests that no longer find new defects (pesticide paradox).

Repository data supports test design: It stores defect ID, module, type, severity, cause, fix and test case reference. Testers use it to select high-risk modules, create test cases for historical defect types and build checklists.

Identifying patterns: Group defects by module, type and origin phase; use Pareto charts and trend graphs. Example: 40% of defects are input-validation defects in the order module, so validation tests become a priority in every release.

Improvements to prevent recurrence: Add a regression test for every fixed defect, perform root-cause analysis, share lessons with developers, update coding and review checklists, and review repository metrics at each release gate.

Question 13

Gaps in testing: Only functional tests were run; no load, stress, spike or endurance testing; unrealistic test data and environments; no analysis of concurrent-user scenarios (for example a thousand students submitting an exam at once).

Impact of test planning and coverage: A plan defines performance objectives, workload models, environment and exit criteria. Without coverage of high-load scenarios, bottlenecks (database connections, memory, server threads) stay hidden until production.

Better strategy improves reliability: Load testing at expected peak, stress testing beyond peak to find breaking points, spike and soak testing, and monitoring of response time, throughput and error rate, so capacity is verified before release.

Enhancements: Use JMeter, LoadRunner or Gatling; derive workload from real usage logs; add performance tests to the CI pipeline; apply caching, load balancing and autoscaling; maintain a production-like test environment.

Question 14

Role of the tester in SDLC: Testers participate in each phase: requirement review, test planning, design review, test case preparation, execution, release and maintenance (regression). They act as quality advocates, not just bug finders.

Impact of early involvement: Ambiguities and untestable requirements are found before coding; cost of defects is lower (a defect fixed at the requirement stage costs a fraction of one found in production); test cases are ready when the build arrives, shortening schedules.

Practical example: While reviewing a requirement 'the system should respond quickly', the tester asks for measurable criteria, and it is changed to 'response within 2 seconds for 500 users'. A vague requirement became testable and prevented a performance defect.

Collaboration changes: Include testers in requirement and design reviews and sprint planning, use three-amigos meetings (business, developer, tester), shared definition of done, joint defect triage, and a shared tracker and continuous integration.

Question 15

Identifying gaps: Compare tests against requirements (traceability matrix) and code (statement/branch coverage). Untested input classes, boundaries, negative cases and rare paths are the gaps.

Proper coverage ensures reliability: When all requirements, inputs classes and paths are exercised, behaviour is shown to be correct across the operating range, not only for typical data.

Improving test case design: Example: discount function for purchase amounts. Partition: below 1,000 (no discount), 1,000-4,999 (5%), 5,000 and above (10%). Add boundaries 999, 1,000, 4,999, 5,000, plus 0, negative and non-numeric. This exposes a '>' versus '>=' defect.

Techniques for better validation: Equivalence partitioning, BVA, decision tables, pairwise testing, white-box coverage (statement, branch, condition), coverage tools (JaCoCo, Cobertura), code reviews and exploratory testing as a supplement.

Question 16

Why delays occur: Reports without steps, vague descriptions, missing environment details, and unclear severity cause back-and-forth and slow triage.

Clear definitions help tracking: When everyone uses the same meaning of error, defect and failure, and of severity versus priority, defects can be classified, assigned, searched and measured consistently.

Structured reports enhance communication: Example: 'Shipment status not updated to Delivered after scan; Build 2.3, Android 13; steps 1-4; expected: Delivered; actual: In Transit; Severity: High; Priority: P1; screenshot and log attached'. The developer can reproduce and fix it immediately.

Reporting standards improvements: Standard template per IEEE 829/1044, mandatory fields, a defect life cycle (New, Assigned, Fixed, Retest, Closed/Reopened), severity-priority guideline, daily triage meetings, and dashboards for ageing defects.

Question 17

Process-based testing: Define one organisation-level test process (planning, design, execution, reporting, closure), based on models such as TMMi or ISO 29119, and apply it to every module.

How standardisation improves quality: Uniform templates, entry/exit criteria and metrics make modules comparable, reduce dependence on individuals, ease reviews and new-joiner training, and ensure no module is under-tested.

Example: Every module must pass the same gates: reviewed test plan, test cases mapped to requirements, 100% high-priority cases executed, no open critical defects, and sign-off report. Module A (login) and Module B (billing) are then tested to the same standard.

Methods to ensure consistency: Test strategy and templates, checklists, peer reviews, a central test management tool, common defect-severity guidelines, process audits and metrics reviews, and shared automation frameworks.

Question 18

Why they were missed: Testing used typical values only, requirements did not specify limits, time pressure, no systematic design technique, and an assumption that valid-looking inputs represent all inputs (violating the principle that exhaustive testing is impossible without smart selection).

How boundary testing helps: Defects cluster at the edges of input ranges because of mistakes in relational operators and loop limits. Testing at min, min-1, min+1, max-1, max, max+1 detects these off-by-one errors.

Including edge cases: Example: password length 8-16. Test 7, 8, 9, 15, 16, 17; also empty, null, special characters, very long strings, leap-day date, zero and negative quantities, and maximum file size.

Improvements in test design: Use BVA and equivalence partitioning together, specify limits in requirements, add error-guessing checklists from past defects, review test cases for edge coverage, and automate them in the regression suite.

Question 19

Applying repository concepts: Introduce a central defect repository (Jira, Bugzilla, Mantis) in which every defect is logged with a unique ID and tracked through its life cycle.

How documentation improves management: Allows tracking of status and ownership, prevents loss and duplicates, supports prioritisation, gives an audit trail and gives evidence for release decisions.

Structured repositories support analysis: Fields such as module, type, severity, phase injected and detected, and cause allow queries and charts (defect density, defect removal efficiency, trend by module). Example: analysis shows that most defects are injected during requirements, so reviews are strengthened.

Documentation practices: Mandatory fields and template, reproducible steps with attachments, linking to test case and requirement, regular clean-up and triage, naming and classification conventions, and training of the team.

Question 20

Redesign using principles: Apply early testing, risk-based prioritisation (transactions are high-risk), defect clustering, regular test refresh (pesticide paradox) and traceable test documentation within a defined test process.

How structured testing improves quality: Requirements are covered by designed test cases, environments and data are controlled, results are recorded, and regression testing protects existing functionality; failures become predictable and preventable.

Example: Transaction processing: tests for debit and credit atomicity (failure after debit must roll back), concurrent transfers on the same account, duplicate request submission, timeout handling, limits and boundary amounts, insufficient funds, and audit log verification.

Techniques to enhance defect detection: BVA and equivalence partitioning, decision tables for business rules, state-transition testing for transaction states (initiated, pending, completed, failed, reversed), ACID-property tests, concurrency, performance and security testing, automated regression and static code reviews.